12. Design Distributed File Storage

Key Terms:

Term	Full Name	What it does
CDC	Content-Defined Chunking	Splits files at content-based boundaries (not fixed size)
SHA-256	Secure Hash Algorithm 256-bit	Hash function for content-addressable storage
Dedup	Deduplication	Storing identical content once, referenced by hash
Delta Sync	Incremental synchronization	Only sync changed blocks, not entire file
EB	Exabyte	1 million terabytes (10^18 bytes)
LSM Tree	Log-Structured Merge Tree	Write-optimized storage engine

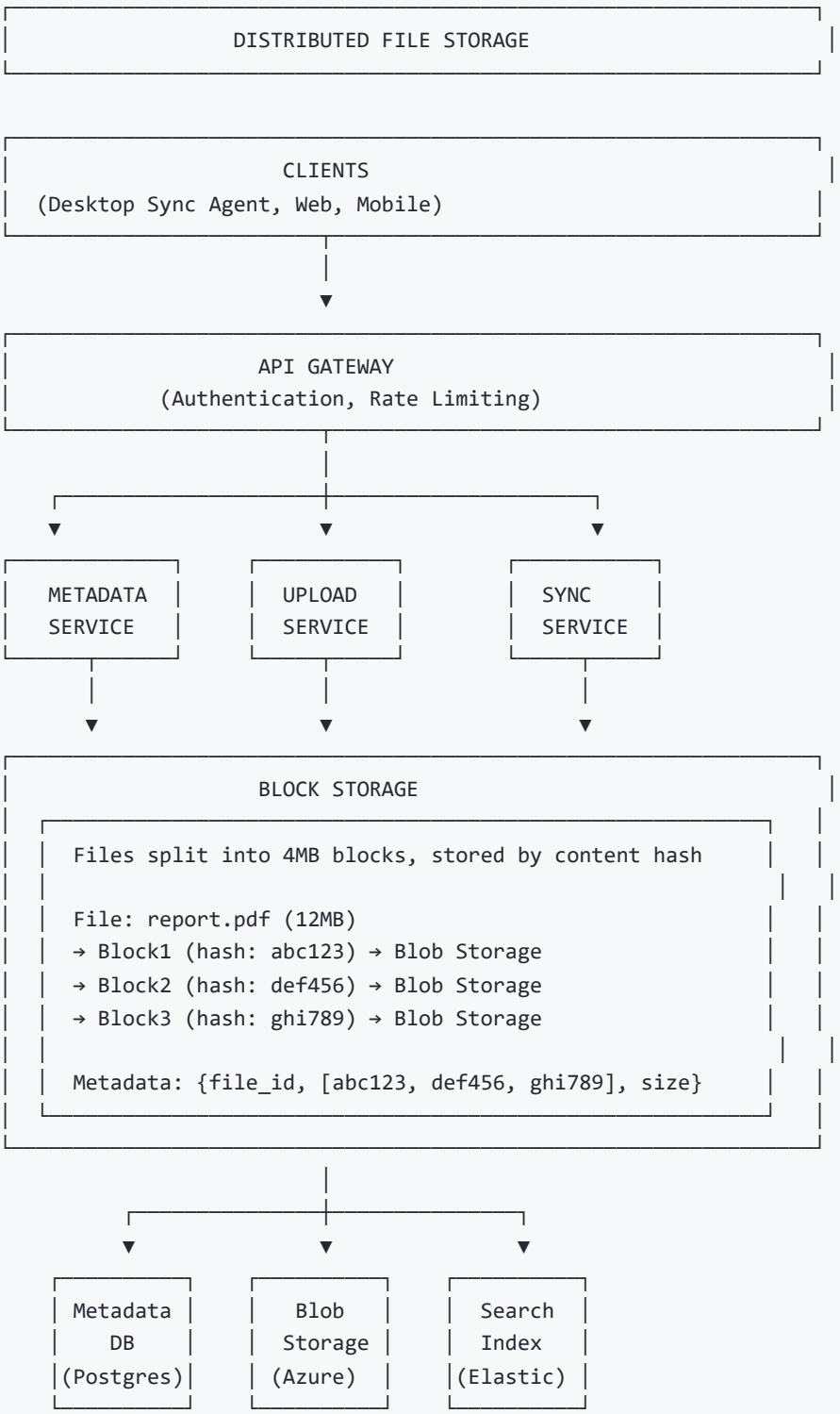
Problem Statement

Design a cloud file storage system like OneDrive or Dropbox that syncs files across devices with versioning and sharing capabilities.

Clarifying Questions to Ask

1. **Scale**: How many users? Total storage? (500M users, 1EB?)
2. **File sizes**: Average and max file size? (avg 1MB, max 50GB?)
3. **Sync latency**: How fast should changes sync? (<5 seconds?)
4. **Versioning**: How many versions to keep? (30 days?)
5. **Sharing**: Public links, user sharing, permissions?
6. **Offline**: Support offline access and sync?
7. **Deduplication**: Dedupe identical files across users?

High-Level Design



API Design

File APIs

Method	Endpoint	Description

`POST`	`/api/v1/files/upload`	Upload a file (multipart)
`POST`	`/api/v1/files/upload/init`	Initialize chunked upload
`PUT`	`/api/v1/files/upload/{uploadId}/chunk/{n}`	Upload a chunk
`POST`	`/api/v1/files/upload/{uploadId}/complete`	Complete chunked upload
`GET`	`/api/v1/files/{fileId}`	Download a file
`GET`	`/api/v1/files/{fileId}/metadata`	Get file metadata
`DELETE`	`/api/v1/files/{fileId}`	Delete a file

Folder APIs

Method	Endpoint	Description
`POST`	`/api/v1/folders`	Create a folder
`GET`	`/api/v1/folders/{folderId}/contents`	List folder contents
`PUT`	`/api/v1/files/{fileId}/move`	Move file to another folder

Sync APIs

Method	Endpoint	Description
`GET`	`/api/v1/sync/changes?since={timestamp}`	Get changes since last sync
`POST`	`/api/v1/sync/conflict/resolve`	Resolve sync conflict

Key Components

Component	Purpose	Technology
Metadata Service	File/folder hierarchy	PostgreSQL
Block Service	Store file chunks	Azure Blob Storage
Sync Service	Track changes, notify	WebSocket + Kafka
Dedup Service	Content-addressable storage	Hash-based lookup
Sharing Service	Permissions, links	PostgreSQL
Search Service	File content search	Elasticsearch

Chunking and Deduplication

```
FILE CHUNKING (Rolling Hash / Rabin Fingerprint):
┌─────────────────────────────────────────────────────────────────┐
│  Instead of fixed 4MB blocks, use content-defined chunking:     │
│                                                                 │
│  File content: [====================]                         │
│  Fixed chunks: [====][====][====][==]  (inefficient for edits) │
│  CDC chunks:   [===][=====][====][===] (boundaries by content)  │
│                                                                 │
│  Benefit: Small edit = only 1-2 chunks change, not all          │
└─────────────────────────────────────────────────────────────────┘

DEDUPLICATION:
┌─────────────────────────────────────────────────────────────────┐
│  Block stored by content hash (SHA-256)                         │
│                                                                 │
│  User A uploads: file1.pdf → blocks [abc, def, ghi]             │
│  User B uploads: file2.pdf → blocks [abc, xyz, ghi]             │
│                                                                 │
│  Storage: [abc, def, ghi, xyz] ← only 4 blocks, not 6           │
│  Savings: ~50% storage reduction in practice                    │
└─────────────────────────────────────────────────────────────────┘
```

Sync Protocol

```
CLIENT-SERVER SYNC:

1. Client tracks local changes (file watcher)
2. Client sends metadata diff to server:
   {
     "changes": [
       {"path": "/docs/file.txt", "action": "modified", "hash": "abc123"}
     ],
     "last_sync": 1705312345
   }

3. Server responds with:
   {
     "upload_required": ["/docs/file.txt"],
     "download_required": ["/photos/new.jpg"],
     "conflicts": []
   }

4. Client uploads changed blocks
5. Server notifies other clients via WebSocket

CONFLICT RESOLUTION:
  - Last-write-wins (simple)
```

```
- Create conflict copy: "file (conflict).txt"
- Let user resolve manually
```

Database Schema

```
-- Files and Folders
CREATE TABLE files (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    parent_id BIGINT,          -- folder hierarchy
    name VARCHAR(255),
    is_folder BOOLEAN,
    size BIGINT,
    content_hash VARCHAR(64), -- for quick comparison
    version INT,
    created_at TIMESTAMP,
    modified_at TIMESTAMP,
    deleted_at TIMESTAMP       -- soft delete for trash
);

-- Block References
CREATE TABLE file_blocks (
    file_id BIGINT,
    block_index INT,
    block_hash VARCHAR(64),    -- content-addressable
    PRIMARY KEY (file_id, block_index)
);

-- Sharing
CREATE TABLE shares (
    id BIGINT PRIMARY KEY,
    file_id BIGINT,
    shared_with_user_id BIGINT,
    permission VARCHAR(20),    -- view, edit
    link_token VARCHAR(64),    -- for public links
    expires_at TIMESTAMP
);
```

Scalability Considerations

- **Chunking**: 4MB chunks balance dedup vs. overhead
- **Metadata sharding**: Shard by user_id
- **Blob storage**: Azure Blob with hot/cool/archive tiers
- **Sync optimization**: Delta sync (only changed blocks)
- **CDN**: Cache popular shared files at edge

Trade-offs

Decision	Option A	Option B	Recommendation
Chunking	Fixed size	Content-defined	**CDC** - better dedup
Sync	Polling	Push (WebSocket)	**Push** - real-time
Conflicts	Auto-merge	User resolve	**Conflict copy** - safe
Storage tiers	Single tier	Hot/cool/archive	**Tiered** - cost savings

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Consistent Hashing	Chunk distribution across storage nodes	Tech Guide
Raft	Consensus for metadata and replication	Tech Guide
LSM Tree	Efficient metadata storage engine	Tech Guide

How to Explain It Well

> **Opening**: "File storage needs efficient sync, deduplication, and versioning. I'll design for 500M users with petabytes of data."

> **Key talking points**:
>
> - "Content-defined chunking: edit 1 byte, upload 1 chunk not whole file"
> - "Deduplication by content hash saves ~50% storage"
> - "WebSocket push for instant sync across devices"
> - "Blob storage tiers: hot for recent, archive for old versions"
> - "Conflict detection via content hash comparison"
